

Computer System Security

Module Leader: Dr. Ghada Elsayed

Group Number: 1

Title: University Social Platform

<i>Name</i>	<i>ID</i>	<i>Role</i>
Belal Mohamed	236990	DevOps & Backend
Haneen Abdulfattah	229914	Frontend
Omar Ahmed	236384	Medical Excuse

Project Scope and Features

The proposed project is an online site that will be a secure social network aimed at university students. The platform enables the students to communicate in a secure and closed setting with their academic identity.

Core Features:

User Authentication

- 1) Customers are able to create and log in with their credentials.
- 2) Handling of secure sessions with tokens.

Profile Management

- 1) There is an option of viewing and managing their personal profiles.
- 2) User information (name, location, profile picture etc.) display.
- 3) After Creation and Interaction.
- 4) Users are able to make text and pictures posts.
- 5) Possibility to comment and like posts.
- 6) The posts are shown dynamically in a feed.

File Sharing

- 1) Users can post and share pictures (limited to some extent).
- 2) Facilitates exchange of academic material like notes and contents.

Friend System

- Users are able to add friends and administer them.
- Display of friend lists
- Chat with them

Responsive Design

- The platform is responsive in both desktop and mobile screens.

Role-Based Features

- No Segregation in Roles is implemented.

Technical Security Approaches

*List the security measures implemented in your project in three sections across the **Frontend**, **Backend**, and **DevOps** layers using bullet points.*

Frontend:

- Secured paths to allow entries to authorized individuals only
- Auto-redirect unauthorized access to login page
- Management of session based on JWT tokens in Redux
- Auto logout in case of unauthorized API response.
- Formidable input validation with Formik and Yup
- Rejection of submissions with empty or all-whitespace content
- Sanitization of the clients to minimize XSS
- Authentication of file uploads (type and size limits)
- Role-based UI display (e.g. only admins can see this)
- UI control based on ownership (only the user can edit his or her posts)
- Authorize API communication with Authorization headers
- Automatic state management to avoid runtime errors
- Predefined test cases of functional security testing Manually

Backend:

- JWT token-based authentication with 24-hour expiration and Bearer token validation
- BCrypt password hashing with 12 salt rounds for secure password storage
- Three-tier rate limiting: Login (5 attempts/15min), Auth (100 requests/15min), General (1000 requests/15min)
- Email format validation with regex and password strength enforcement (minimum 8 characters)
- Content length validation: posts (5000 chars), comments (1000 chars), messages (2000 chars) to prevent buffer memory from being filled
- NoSQL injection prevention using express-mongo-sanitize middleware
- HTTP Parameter Pollution (HPP) prevention middleware
- File upload security with type restrictions (JPEG/JPG/PNG/GIF), 5MB size limit, and filename sanitization
- Authorization checks ensuring users can only modify their own data (posts, comments, messages)
- CORS configuration with dynamic whitelist from environment variables and credentials support
- Helmet middleware for security headers (XSS)
- Environment variable management with dotenv for secrets (JWT_SECRET, MONGO_URL)
- Password field exclusion from all API responses using .select('-password')
- Morgan HTTP request logging for audit trails
- Input sanitization and trimming for all user-provided data
- Timestamp-based unique file naming to prevent filename collisions and path traversal attacks

DevOps:

- Non-root user execution in all containers (nodejs user with UID 1001, GID 1001)
- Container health checks with wget for server (/health endpoint) and client (/) endpoints
- Docker image vulnerability scanning with Trivy for CRITICAL/HIGH/MEDIUM severity issues
- Automated npm audit on both server and client for dependency vulnerabilities using high severity threshold
- TruffleHog secret scanning with --only-verified flag to detect hardcoded credentials
- GitLeaks secret scanning for additional secret detection in repository history
- Snyk dependency vulnerability scanning with high severity threshold
- CodeQL static application security testing (SAST) for JavaScript code analysis
- License compliance checking with automated validation of dependency licenses
- GitHub Actions CI/CD pipeline with security-scan, test, and docker-build jobs
- Scheduled weekly security scans (every Monday at 9 AM UTC)
- Container restart policy: unless-stopped for automatic recovery
- Isolated Docker network using bridge driver for communication
- Required environment variables validation (MONGO_URL and JWT_SECRET must be set)
- Health check dependencies ensuring server is healthy before starting client
- Volume persistence for uploaded assets (/app/public/assets)
- Security labels in Dockerfiles with maintainer and version information
- .dockerignore files to prevent sensitive files from being included in images

Project Demonstration Video

Provide a link (from Google drive(make it accessible)) to a comprehensive video or a Video demonstrating the full implementation and functionality of your application.

GUI WALKTHROUGH:

https://drive.google.com/file/d/1KpN1G6vgPEQ1Z_IYR7EnO_heGR0OInt/view?usp=sharing

DEVOPS IMP WALKTHROUGH:

<https://drive.google.com/file/d/1KOV8OD3OXwEuezuAVD9JPvcwAyBsaV0M/view?usp=sharing>

BACKEND IMP WALKTHROUGH:

<https://drive.google.com/file/d/1MZCjaMkzn6pARcqtYI5dpdx6QR4MVxP9/view?usp=sharing>

FRONTEND IMP WALKTHROUGH: